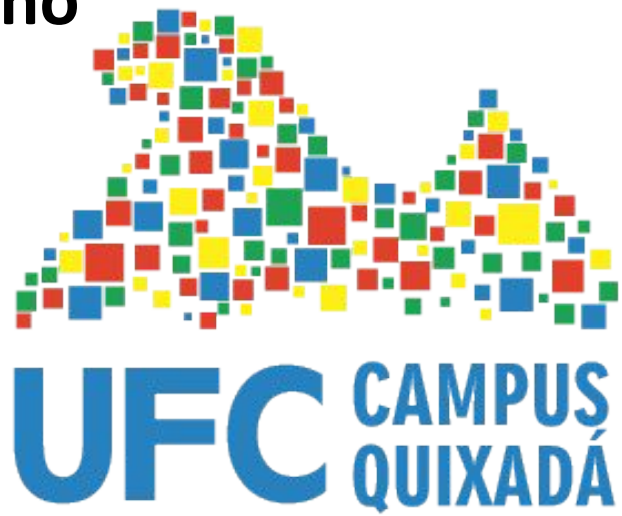
Introdução ao JavaScript



Prof. Sidartha Carvalho



AULA 1/4

JS: Introdução

O Trio do Desenvolvimento Web

HTML

Define o conteúdo das páginas web

CSS

Especifica o design das páginas web

JavaScript (JS)

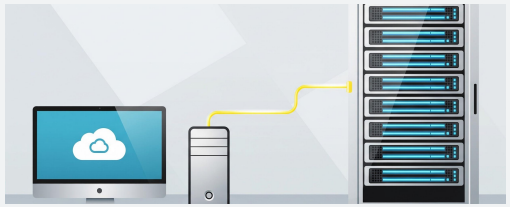
Fornece "comportamento" às páginas

O Poder do JavaScript Além da Web

- Não é usado somente em páginas Web
- Presente em aplicações desktop e servidores

Exemplos de Ecossistema:

NodeJS (servidor), MongoDB, CouchDB, etc



JS: Introdução ao JavaScript

JS ≠ Java

Apesar do nome similar, surgiu em 1995 aproveitando a popularidade do Java na época como estratégia de marketing.

Origens e Padrão

- Criada por **Brendan Eich** em 1995.
- Padronizada em 1997.
- **ECMAScript** é o nome oficial.

JS no Contexto Web

- Manipular componentes **HTML/CSS**.
- Alterar estilos visualmente.
- Adicionar **comportamento** e eventos.



JS: alterando HTML

```
<!DOCTYPE html>
<html>
<body>

<h2>O que posso fazer com JavaScript?</h2>

<p id="meup">JavaScript pode alterar o conteúdo de um elemento HTML.</p>

<button type="button" onclick='document.getElementById("meup").innerHTML =
 "Novo texto usando JavaScript!"'>Click Me!</button>

</body>
</html>
```

JS: alterando HTML

```
<!DOCTYPE html>
<html>
<body>

<p>JavaScript pode alterar os atributos dos elementos HTML.</p>

<p>Vamos alterar o endereço de uma imagem de forma dinâmica.</p>

<button onclick="document.getElementById('myImage').src='pic_bulbon.gif’”>Li
gar lâmpada</button>

<img id="myImage" src="pic_bulboff.gif" style="width:100px">

<button onclick="document.getElementById('myImage').src='pic_bulboff.gif’”>A
pagar a lâmpada</button>

</body>
</html>
```

JS: alterando CSS

```
<!DOCTYPE html>
<html>
<body>

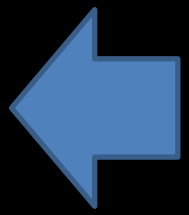
<p id="demo">JavaScript pode alterar o estilo de um elemento HTML.
</p>

<button type="button" onclick="document.getElementById('demo').sty
le.fontSize='35px'">Clique aqui!</button>

</body>
</html>
```

JS: <script>: alert()

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Aula de JS</title>
    <script>
        alert("Olá, Mundo!");
    </script>
</head>
<body>
    <h1>JavaScript</h1>
    <h2>Linguagem de programação</h2>
</body>
</html>
```



A tag <script> é usada para inserir código JS, pode ser inserida no <head>, no <body> ou em arquivo específico.

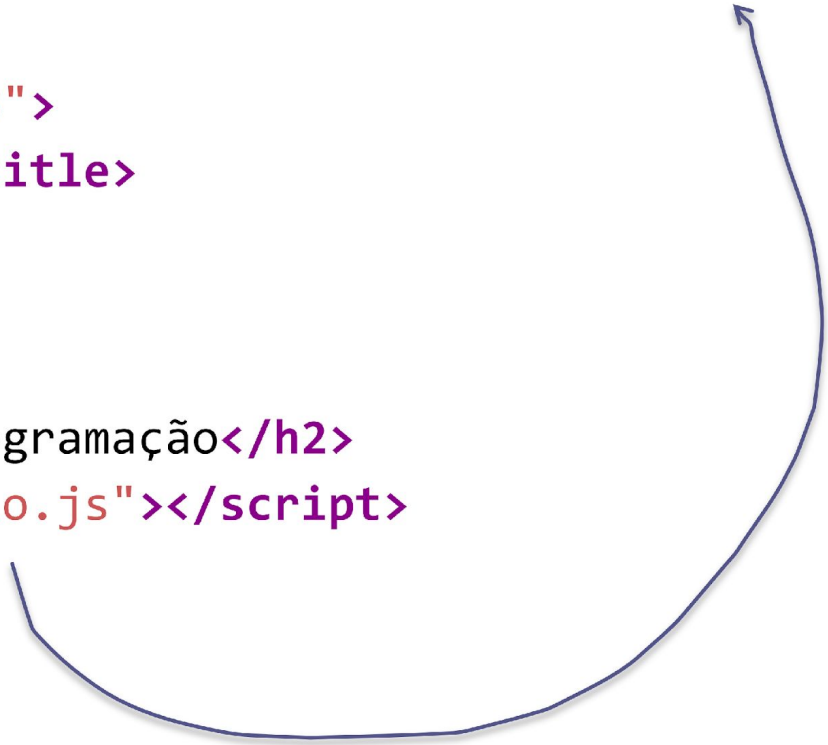
JS: <script>

Externalizando...

```html
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Aula de JS</title>
  </head>
  <body>
    <h1>JavaScript</h1>
    <h2>Linguagem de Programação</h2>
    <script src="js/hello.js"></script>
  </body>
</html>
```

js/hello.js

```
alert("Olá, Mundo!");
```



JS: Calculadora

```
<html>
<head><title>Calculadora</title></head>
<body>
<form id="calc">
<input id="n1" /> + <input id="n2" />
<button id="botao">OK</button>
</form>
<p id="resultado"></p>
<script>
function exibe() {
res.textContent = parseFloat(n1.value) +
parseFloat(n2.value);
return false;
}
var n1 = document.getElementById('n1');
var n2 = document.getElementById('n2');
var res = document.getElementById('resultado');
botao.onclick = exibe;
</script>
</body>
</html>
```

Estrutura HTML

Define os campos de entrada (n1, n2) e o botão de ação para coletar os dados do usuário.

Lógica JS

Captura os valores, realiza a conversão para número (parseFloat) e exibe a soma no DOM.



Manipulação de DOM

JS: console.log()

O que é?

Função usada para imprimir dados no console.

- Depuração e erros
- Acompanhamento de execução
- Informações importantes

Como usar?

Dentro de tags <script>, inclua a chamada:

```
console.log('teste')
```

Como visualizar?

Abra o **Modo Desenvolvedor** no seu navegador:

- **Chrome:** Control + Shift + C
- **Firefox:** Control + Shift + K

JavaScript: Comentários

Linha Única

Utilizado para notas curtas ou desativar código temporariamente. Inicia com duas barras inclinadas.

```
// Comentário simples
```

Múltiplas Linhas

Ideal para documentação extensa ou blocos de comentários explicativos.

```
/* Este comentário
abrange várias
linhas... */
```



JS: tipos de dados

Tipagem dinâmica: o tipo é atribuído a partir do valor da variável.

Dados Primitivos

- **Number:** Valores numéricos.
- **String:** Sequências de caracteres.
- **Boolean:** true ou false.
- **Undefined:** Variável sem valor.

Dados Complexos

- **Object:** Coleção de propriedades.
  - Null: Ausência intencional de valor.
- **Function:** Blocos de código executáveis.



JS: expressões



JS avalia expressões da esquerda para a direita

```
var x = 16 + 4 +
"Volvo";
// Resultado: "20Volvo"
```

```
var x = "Volvo" + 16 + 4;
// Resultado: "Volvo164"
```

Para evitar problemas, use sempre os parênteses:

```
(16 + 4) + "Volvo"
```

JS: tipos de dados



Undefined

```
var x;
```

Quando uma variável é declarada mas não recebe valor, seu tipo inicial no JavaScript é undefined.



Number

```
x = 5;
```

Ao atribuir um número à variável, o JavaScript dinamicamente altera seu tipo para Number.



String

```
x = "John";
```

Valores entre aspas transformam o tipo da variável para String, demonstrando a tipagem dinâmica.

JS: tipos de dados



Conversão de string para número (defina a base)

```
parseInt("123", 10)
```

123

Base Decimal

```
parseInt("010", 10)
```

10

Base Decimal

```
parseInt("11", 2)
```

3

Base Binária

JS: String

Comprimento

```
"hello".length
```

5

Acesso a Caractere

```
"hello".charAt(0)
```

'h'

Substituição

```
"hello, world".replace("hello", "goodbye")
```

"goodbye, world"

Transformação

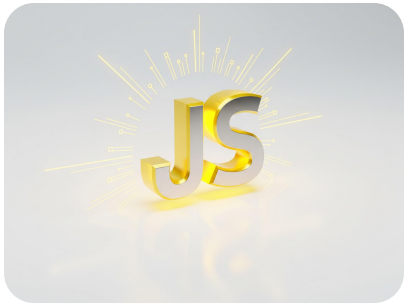
```
"hello".toUpperCase()
```

"HELLO"

Strings

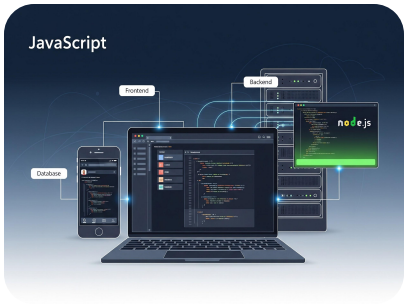
String	length	Returns the number of characters in a string
	concat( )	Joins two or more strings
	indexOf( )	Returns the position of the first occurrence of a specified string value in a string
	lastIndexOf( )	Returns the position of the last occurrence of a specified string value, searching backward from the specified position in a string
	match( )	Searches for a specified string value in a string
	replace( )	Replaces some characters with others in a string
	slice( )	Extracts a part of a string and returns the extracted part in a new string
	split( )	Splits a string into an array of strings
	substring( )	Extracts the characters in a string between two specified indexes
	toLowerCase( )	Displays a string in lowercase letters
	toUpperCase( )	Displays a string in uppercase letters

Resumo da Aula



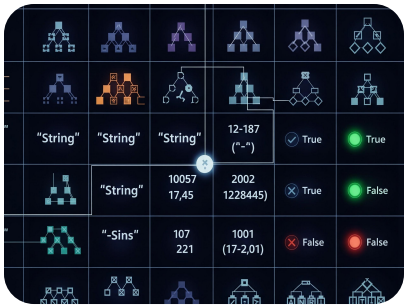
Introdução ao JavaScript

JavaScript é uma linguagem de programação versátil e essencial para a web moderna, permitindo a criação de interfaces dinâmicas e interativas que melhoram a experiência do usuário.



Usos e Aplicações

Desde o desenvolvimento front-end com frameworks modernos até aplicações no servidor com Node.js, o JavaScript está presente em quase todos os domínios da tecnologia atual.



Tipos de Dados e Expressões

Entender como manipular tipos primitivos, objetos e expressões lógicas é a base fundamental para construir algoritmos eficientes e lógicos em qualquer aplicação.

JS: Arrays

```
let cars = ["Saab", "Volvo", "BMW"];
```



[0] "Saab"



[1] "Volvo"



[2] "BMW"

Arrays são variáveis especiais que podem conter mais de um valor por vez. Em JavaScript, os arrays são indexados a partir de zero, permitindo organizar dados relacionados de forma eficiente e acessível.

JS: Objects

An **object** is a collection of related data and/or functionality. These usually consist of several variables and functions, which are called properties and methods when they are inside objects.

```
// Defining an Object

let person = {
firstName: "John",
lastName: "Doe",
age: 50,
eyeColor: "blue"
};

console.log(person);
```

Object Structure

Key: firstName → **Value:** "John"

Key: lastName → **Value:** "Doe"

Key: age → **Value:** 50

Key: eyeColor → **Value:** "blue"



JS: variáveis

Palavra-chave let

Utilizada para declarar variáveis com escopo de bloco.

```
let a;
let name = "simon";
```

Valor Inicial

Variáveis declaradas sem atribuição recebem o valor:

`undefined`



Atenção: Variáveis Globais

Omitir **let** ou **var** torna a variável global. **Evite esta prática!**

```
var mensagem = "Olá mundo";
console.log(mensagem);
```

JS: conversão de tipos

String para Inteiro / Float

```
let textoInteiro = "10";
let inteiro = parseInt(textoInteiro); // 10
let float = parseFloat("10.22"); // 10.22
```

Número para String (Formatado)

```
let milNumber = 1000;
let milString = milNumber.toFixed(2); // "1000.00"
```

Dica de Performance:
Use `parseInt()` para converter strings em números inteiros e `toFixed()` para formatar casas decimais, retornando sempre uma **String**.

JS: operadores

Numéricos: Operações matemáticas básicas (+, -, *, /) e o operador de resto (%).

Atribuição Encurtada: Combina operação e atribuição em um único passo (=, +=, -=, *=, /=, %=).

Incremento e Decremento (++, --):

- **Prefixado (++x):** Altera o valor e depois retorna o novo valor (Ex: se x=3, retorna 4).
- **Pósfixado (x++):** Retorna o valor atual e depois altera a variável (Ex: se x=3, retorna 3 e então x vira 4).

JS: operadores

Concatenação de Strings

Une textos usando o operador **+**. `"hello" + " world"` resultará em **"hello world"**.

Coerção de Tipos

O JS converte números em strings automaticamente durante a soma se houver uma string presente:

- `"3" + 4 + 5` → **"345"** (avalia da esquerda para a direita)
- `3 + 4 + "5"` → **"75"** (soma os números antes da string)

Dica Pro:

Adicionar uma string vazia (`""`) a qualquer valor força a sua conversão imediata para o tipo **String**.

JS: comparativos

Básicos: Para números e strings use `<`, `>`, `<=`, `>=`

Igualdade (==)

Coerção de Tipos:

Converte os valores para o mesmo tipo antes de comparar.

```
"dog" == "dog" // true
1 == true // true
```

Flexível, mas pode causar bugs inesperados.

Identidade (===)

Rigoroso:

Compara o valor e o tipo. Sem conversão automática.

```
1 === true // false
true === true // true
```

Recomendado para evitar erros de lógica.

JS: while e do..while

while

Testa a condição **antes** de executar o bloco. Se for falsa no início, o código nunca será executado.

```
while (true) {
// loop infinito!
}
```



Pré-teste:
0 ou mais iterações.

do..while

Executa o bloco **primeiro**, depois testa a condição. Garante ao menos uma execução.

```
do {
var input = get_input();
} while (isNotValid(input))
```



Pós-teste:
1 ou mais iterações.

Iterações em JavaScript: O Laço for

for (Clássico)

```
for (var i = 0; i < 5; i++) {
// Executa 5 vezes
}
```

for...of

```
for (const num of lista) {
console.log(num);
}
```

for...in

Itera sobre as **Chaves** (nomes das propriedades ou índices).

Ideal para objetos e extração de chaves de propriedades.

for...of

Itera sobre os **Valores** (o conteúdo real dos dados).

Ideal para arrays, strings e coleções iteráveis.

JS: switch

Standard Syntax

```
switch (action) {
  case 'draw':
    drawit();
    break;

  case 'eat':
    eatit();
    break;

  default:
    donothing();
}
```

Comparações usam Igualdade Estrita

A instrução switch usa o operador `= = =` para corresponder aos casos, garantindo que tanto o valor quanto o tipo devam coincidir.

Expressions are Allowed

```
switch (1 + 3) {
  case 2 + 2:
    yay();
    break;

  default:
    neverhappens();
}
```

Operadores Lógicos: && (Short-circuit)

&& (AND) e **|| (OR)** executam o segundo operando dependendo do resultado do primeiro. Isso é conhecido como avaliação de curto-circuito.

Exemplo: Checagem de Objetos

```
var name = o && o.getName();

// Se 'o' for true (não null ou undefined):
Retorna o.getName()

// Se 'o' for false:
Retorna o (false)
```

Tabela de Verdade & Retornos

```
a1 = true  && true       // t && t retorna true
a2 = true  && false      // t && f retorna false
a3 = false && true       // f && t retorna false
a4 = false && (3 == 4)   // f && f retorna false
a5 = 'Cat' && 'Dog'      // t && t retorna "Dog"
a6 = false && 'Cat'      // f && t retorna false
a7 = 'Cat' && false      // t && f retorna false
a8 = ''    && false      // f && f retorna ""
a9 = false && ''         // f && t retorna false
```

JS: Operador Lógico OU (||)

Lógica de Curto-Circuito: a || b

Retorna o primeiro valor **Verdadeiro (Truthy)** encontrado. Se todos forem **Falsos (Falsy)**, retorna o último valor.

Valores Falsy em JS: `false, null, NaN, 0, "", undefined`

```
// Casos Booleanos
o1 = true || true → true
o2 = false || true → true
o3 = true || false → true
o4 = false || (3==4) → false

// Casos de String
o5 = 'Cat' || 'Dog' → "Cat"
o6 = false || 'Cat' → "Cat"
```

```
// Casos Falsy
o7 = 'Cat' || false → "Cat"
o8 = '' || false → false
o9 = false || '' → ""
```

Dica: O JS para a busca assim que encontra o primeiro valor verdadeiro.

JavaScript: Operador Ternário

O operador ternário é uma forma compacta de escrever uma estrutura condicional `if...else`.

Sintaxe Geral

```
condição ? valor_se_verdadeiro : valor_se_falso
```

Exemplo Prático

```
var allowed = (age > 18) ? "yes" : "no";
```

Se Verdadeiro (?)

O código após o ponto de interrogação é executado se a condição for **true**.

Se Falso (:)

O código após os dois pontos é executado se a condição for **false**.

JS: exceções

```
try {
// Código que pode falhar
} catch(erro) {
// Lida com o erro
} finally {
// Executa sempre
}

throw new Error("Erro!");
```



Gestão de Erros no JS

Robustez e confiabilidade no fluxo de execução do software.

O Fluxo de Exceções

TRY: Define um bloco de código para ser testado quanto a erros enquanto está sendo executado. É o ambiente vigiado onde as operações críticas ocorrem.

CATCH: Captura o erro gerado no try. Permite que o programa continue rodando graciosamente em vez de travar completamente o sistema.

FINALLY: Garante a execução de limpeza (como fechar conexões) independente do sucesso ou falha das operações anteriores.

THROW: Permite criar erros personalizados. Você define quando algo é "errado" para a lógica do seu negócio, disparando uma exceção manual.

JS: Tratamento de Exceções

Estrutura Try-Catch-Finally

O mecanismo para lidar com erros em tempo de execução:

- **try**: Bloco onde o código é testado.
- **catch**: Captura o erro para tratamento.
- **finally**: Executa sempre, ocorrendo erro ou não.

A Cláusula Throw

Você pode criar seus próprios erros personalizados:

- Pode lançar Strings, Numbers ou Objetos.
- **new Error()**: Cria um objeto de erro com mensagem e pilha de chamadas (stack trace).
- Interrompe a execução normal até encontrar um catch.

EXEMPLO

```
// Lançando e capturando
try {
    if(v < 0) throw "Negativo";
}catch (err){
        console.log("Erro: " + err);
} finally {
    console.log("Fim do teste");
}
```

JSON

JavaScript Object Notation

Usado para serializar objetos em formato legível ao ser humano.

```
{ "Books":
  [
    { "ISBN":"ISBN-0-13-713526-2",
      "Price":85,
      "Edition":3,
      "Title":"A First Course in Database Systems",
      "Authors":[ {"First_Name":"Jeffrey", "Last_Name":"Ullman"},
                  {"First_Name":"Jennifer", "Last_Name":"Widom"} ] }
    ,
    { "ISBN":"ISBN-0-13-815504-6",
      "Price":100,
      "Remark":"Buy this book bundled with 'A First Course' - a great deal!",
      "Title":"Database Systems:The Complete Book",
      "Authors":[ {"First_Name":"Hector", "Last_Name":"Garcia-Molina"},
                  {"First_Name":"Jeffrey", "Last_Name":"Ullman"},
                  {"First_Name":"Jennifer", "Last_Name":"Widom"} ] }
  ]
```

JSON: Estrutura de Dados

Valores Base

- Tipos Primitivos:
  - Número, string, booleano, null
- Objetos { }:
  - Conjuntos de pares chave-valor
- Arrays [ ]:
  - Listas ordenadas de valores

Exemplo Prático

```
{
"alunos": [
     { "nome": "João", "notas": [8, 9, 7] },
     { "nome": "Maria", "notas": [8, 10, 7] },
     { "nome": "Pedro", "notas": [10, 10, 9] }
     ]
}
```

Por que usar JSON?

O formato JavaScript Object Notation é leve, de fácil leitura para humanos e simples de processar para máquinas, tornando-se o padrão ouro para troca de dados na web.

JS: Acesso aos Atributos

Dot Notation (.)

```
obj.name = "Simon";

var name = obj.name;
```

Bracket Notation ([])

```
obj["name"] = "Simon";

var name = obj["name"];
```

Ambas as formas são equivalentes

- A notação de colchetes utiliza **strings**.
- Permite acesso dinâmico em **tempo de execução** e uso de **palavras reservadas**.

Exemplo de Necessidade: Atributos Dinâmicos ou Reservados

```
var property = "first-name";
obj[property] = "Simon"; // Única forma para chaves com hífen ou variáveis
obj["class"] = "A1"; // Útil para palavras reservadas
```

JS: Acesso Seguro a Atributos

```
// Definição do Objeto
const usuario = {
    nome: "Ana",
    preferencias: {
        tema: "dark"
    }
};

// Acesso com Optional Chaining (?.)
// E Nullish Coalescing (??)
const rua = usuario?.endereco?.rua ??
"N/A";

// Resultado
console.log(rua); // "N/A"
```

Encadeamento Opcional (?.)

Permite ler o valor de uma propriedade sem validar cada referência na cadeia.

Se for `null` ou `undefined`, retorna undefined sem erro.

Coalescência Nula (??)

Retorna o operando direito quando o esquerdo é null ou undefined.

Ideal para definir valores padrão de forma segura.

JS: Sintaxe de Objetos

```
var obj = {
  name: "Carrot",
  "for": "Max",
  details: {
    color: "orange",
    size: 12
  }
};
```

Acesso por Ponto (Dot Notation)

```
obj.details.color
// Retorna: "orange"
```

Acesso por Colchetes (Bracket Notation)

```
obj["details"]["size"]
// Retorna: 12
```

JavaScript: Iteração de Objetos

Em JavaScript, podemos percorrer todas as chaves de um objeto utilizando a estrutura **`for...in`**.

```
var obj = { 'name': 'Simon', 'age': 25 };

for (var attr in obj) {
console.log(attr + ' = ' + obj[attr]);
}
```

Dica Importante:

A variável **`attr`** assume o nome de cada propriedade (chave) do objeto em cada iteração, permitindo acessar o valor correspondente via **`obj[attr]`**.

JS: arrays

Tipo especial de objeto onde as chaves são números e não strings.

```
// Constructor syntax
var a = new Array();
a[0] = "dog";
a[1] = "cat";
a[2] = "hen";
a.length // 3

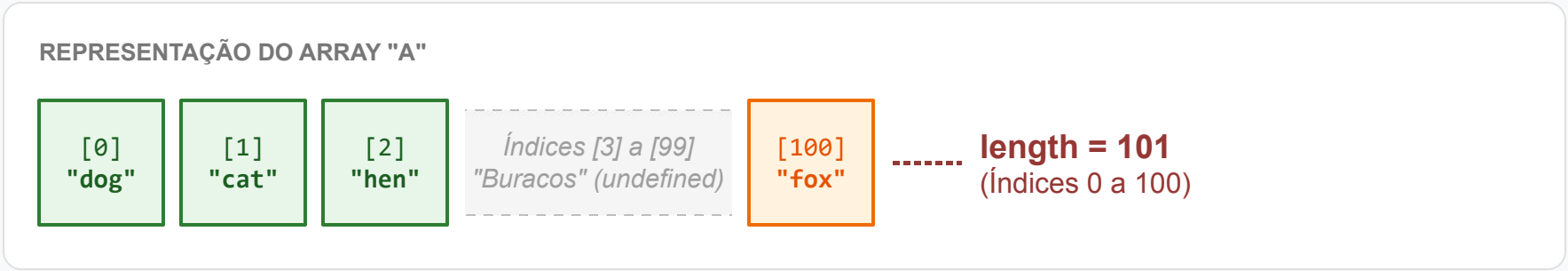
// Literal syntax:
var a = ['dog', 'cat', 'hen']
```

```
// Adding elements:
var palavras = ["UFC",
"Ensino"];
palavras.push("Inovação");
// add nova palavra ao array

// Accessing:
palavras[0] // "UFC"
palavras[2] // "Inovação"
```

JS: Arrays Dinâmicos e Esparsos

Em JavaScript, arrays crescem automaticamente ao atribuir valores a índices altos, preenchendo o intervalo com "buracos" (empty slots).




```
var a = ["dog", "cat", "hen"];
// length: 3

a[100] = "fox";
// Expande o array para 101 itens

a.length; // 101

typeof(a[90]); // "undefined"
```

Conceito Chave:

Ao definir um índice distante, o JS reserva o espaço, mas não inicializa os intermediários. **Índices vazios não existem fisicamente na memória até serem atribuídos.**

Boa Prática (Append Seguro):

`a[a.length] = item;`

Adiciona ao final sem deixar buracos.

JS: arrays

length, concat(), join()

O atributo **length** retorna o número de elementos. **concat()** une arrays e **join()** converte os membros em uma string única com delimitador.

pop() & push()

pop() remove e retorna o último elemento do array. **push()** adiciona novos membros ao final do array e retorna o novo comprimento total.

shift() & unshift()

shift() remove o primeiro elemento da lista. Já o **unshift()** adiciona membros ao início do array, deslocando os índices existentes.

reverse() & sort()

reverse() inverte a ordem dos elementos. **sort()** organiza os elementos do array (por padrão, em ordem alfabética ou crescente).

slice() & splice()

slice() seleciona parte do array sem modificá-lo. **splice()** adiciona ou remove elementos em qualquer posição específica.

Resumo de Operações

Estes métodos fundamentais do JavaScript permitem a manipulação dinâmica de coleções de dados de forma eficiente e programática.

JS: funções de manipulação de arrays



Map

Cria um novo array transformando cada elemento do original.

```
[1, 2, 3] → [2, 4, 6]
```



Filter

Filtra elementos com base em uma condição, retornando apenas os verdadeiros.

```
[1, 2, 3] → [2, 3]
```



Reduce

Reduz o array a um único valor acumulado (como uma soma).

```
[1, 2, 3] → 6
```

JS: Array Manipulation

.map() method

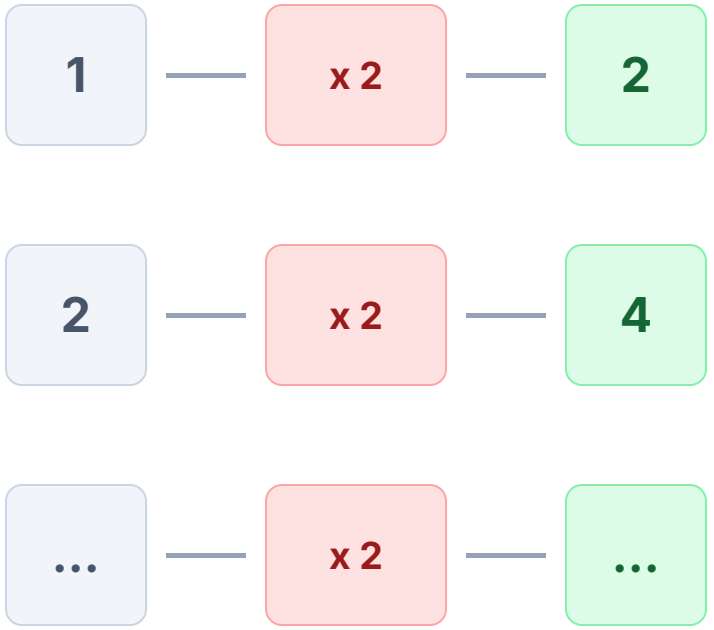
Cria um novo array preenchido com os resultados da chamada de uma função fornecida em cada elemento do array de chamada.

```
// Original Array
const numbers = [1, 2, 3, 4];

// Transformation
const doubled = numbers.map(n ⇒ n
* 2);

// Result
console.log(doubled);
// [2, 4, 6, 8]
```

Visual Flow



JS: Array Manipulation

`.filter()` Cria um novo array com elementos que passam em um teste específico.

```
const numbers = [1, 2, 3, 4];
const evens = numbers.filter(item => item % 2 === 0);
console.log(evens); // [2, 4]
```

FLUXO DE EXECUÇÃO



O Método .reduce()

O método **reduce** é utilizado para reduzir um array a um único valor. Ele executa uma função de callback em cada elemento, passando o resultado da computação anterior para o próximo passo. É ideal para somas, médias ou transformar arrays em objetos.

Arrow Function (Conciso)

```
const nums = [1, 2, 3, 4];

const soma = nums.reduce((acc, val) => acc + val, 0);

// Muito mais legível e eficiente visualmente.
```

JS: Funções de Manipulação de Arrays

Transformando Array em Objeto

O método **.reduce()** é extremamente versátil, permitindo converter uma lista de elementos em um objeto estruturado, utilizando um objeto vazio como valor inicial.

Exemplo: Mapeando Ids para Nomes

```
const users = [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Bob' }
];

const userMap = users.reduce((acc, user) => {
  acc[user.id] = user.name;
  return acc;
}, {});

// Resultado: { 1: 'Alice', 2: 'Bob' }
```

JS: Destructuring Arrays

Forma resumida de acessar elementos de um array

Abordagem Comum

```
const winner = scores[0];
const secondwinner =
scores[1];
```

Com Destructuring

```
const [gold, silver] = scores;
const [gold, silver,
...outros] = scores;
```

```
const scores = [90, 85, 80, 75, 70];

console.log(gold); // Output: 90
console.log(silver); // Output: 85
console.log(outros); // Output: [80, 75, 70]
```

JS: Destructuring Objects

Acesso Tradicional

```
const nome = user.nome;
const idade = user.idade;
```

Forma Resumida (Destructuring)

```
const { nome, idade } = user;
```

- Vincula automaticamente os atributos do objeto às variáveis com os mesmos nomes.

Renomeando Variáveis

```
const { nome: primeiroNome } = user;
```

- Extrai `user.nome` e atribui o valor à nova variável `primeiroNome`.

JS: Destructuring Objects

1. Valores Default

Se um atributo estiver ausente em alguns objetos, você pode definir um valor padrão diretamente na desestruturação.

```
const {nome, idade, filhos = 0}
```

2. Desestruturação em Parâmetros

Evita a necessidade de acessar propriedades manualmente dentro da função, tornando o código mais limpo e legível.

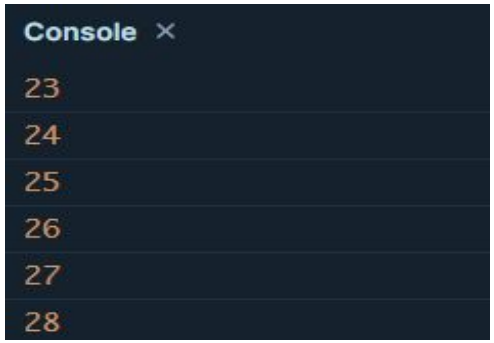
```
function fullName({firstName, lastName})
{
  console.log(firstName);
  console.log(lastName);
}

const user = { firstName: 'Sidartha',
... };
fullName(user);
```

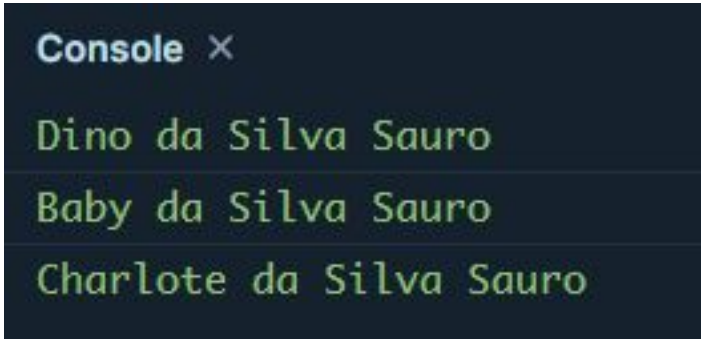
Resumo:
arrays, objetos, variáveis, conversão de tipos, operadores e comparativos, JSON.

EXERCÍCIO 1

- 1. Crie um script que instancia um array com 30 posições e itera sobre ele, salvando em cada elemento o valor da posição dele no array somado de 23. Após isso, imprima no console todos os elementos do array.
  - Use o for convencional e o “for of”
  - Deve imprimir do 23 ao 53.



- 2. Dado o array “let nomes = ['Dino', 'Baby', 'Charlote’]” percorra todos os elementos e adicione o sobrenome “ da Silva Sauro” usando a função Map.
  - Resultado:



EXERCÍCIO 2

- Você está desenvolvendo um aplicativo para uma loja online. Um dos requisitos é exibir o preço com desconto de cada produto listado no inventário. A lista de produtos está disponível como um array de objetos, onde cada objeto possui os atributos nome e preço.

- Dado o array de produtos abaixo, escreva um código JavaScript que utilize o método **map** para criar um novo array onde cada produto possui um novo atributo precoComDesconto (10% de desconto aplicado). O array resultante deve manter os outros atributos inalterados.

```
const produtos = [
  { nome: "Camiseta", preco: 50 },
  { nome: "Calça", preco: 100 },
  { nome: "Tênis", preco: 200 },
];
```

EXERCÍCIO 3

- **Você está criando uma funcionalidade para um sistema de biblioteca.**

- É necessário listar apenas os livros que estão disponíveis para empréstimo.

- Os livros têm as informações título, autor e disponível.

- Com base no array de livros abaixo, escreva um código JavaScript que utilize o método **filter** para retornar um novo array contendo apenas os livros com o atributo disponível igual a true.

```
const livros = [
  { titulo: "JavaScript para Iniciantes", autor: "João Silva", disponivel: true
},
  { titulo: "CSS Avançado", autor: "Maria Oliveira", disponivel: false },
  { titulo: "React Rápido", autor: "Ana Souza", disponivel: true },
];
```

EXERCÍCIO 4

- Você está desenvolvendo uma funcionalidade de relatório financeiro para um sistema de vendas. É necessário calcular o valor total das vendas realizadas em um determinado mês.

- Dado o array de vendas abaixo, escreva um código JavaScript que utilize o método **reduce** para calcular e retornar o valor total das vendas.

```
const vendas = [
  { produto: "Notebook", preco: 2500, quantidade: 2 },
  { produto: "Smartphone", preco: 1500, quantidade: 3 },
  { produto: "Teclado", preco: 200, quantidade: 5 },
];
```

EXERCÍCIO 5: Desafio

Você recebeu uma estrutura de dados JSON com a lista de alunos, notas e status de matrícula. Sua tarefa é processar esses dados para gerar um relatório final, aplicando uma sequência de transformações.

```
const alunosJSON = [
{ id: 101, nome: "João", notas: [3, 5], status: "ativo" },
{ id: 102, nome: "Maria", notas: [3, 5, 6], status: "inativo" },
{ id: 103, nome: "Pedro", notas: [4, 6], status: "ativo" },
{ id: 104, nome: "Ana", notas: [7, 9], status: "ativo" }
];
```

Passos para a função gerarRelatorio(alunos):

- **Tratamento de Exceções:** Utilize try...catch...finally. No início do bloco try, verifique se o array possui elementos. Se vazio, use throw new Error("A lista de alunos está vazia!").
- **Filtragem (filter):** Mantenha apenas os alunos com o status igual a "ativo".
- **Transformação e Desestruturação (map e reduce):** Use .map() e desestruture id, nome e notas. Dentro do map, utilize .reduce() para somar as notas e calcule a média.
- **Operador Ternário:** Crie a variável situação ("Aprovado" se média >= 7, senão "Reprovado"). Retorne um novo objeto: { id, nome, media, situacao }.
- **Mapeamento Final (reduce):** Transforme o array resultante em um objeto de dicionário usando o id como chave e o nome como valor (Ex: { 101: 'João', 103: 'Pedro' }).
- **Finalização:** Exiba erros no catch e "Processamento concluído" no finally.

AULA 2/4

JS: function

```
function add(x, y) {
    var total = x + y;
    return total;
}

Se nada for explicitamente retornado, o valor de retorno é
undefined.

Passagem de parâmetros:
    > add()
    NaN // You can't perform addition on undefined
    > add(2, 3, 4)
    5 // added the first two; 4 was ignored
```

JS: function

```
function add() {
    var sum = 0;
    for (var i = 0, j = arguments.length; i < j; i++) {
        sum += arguments[i];
    }
    return sum;
}
    > add(2, 3, 4, 5)
14
```

JS: function expression (funções anônimas)

- Acontece quando uma função é criar sem ter um nome

```
var somaDoisNumeros = function (numero1, numero2) {
    return numero1 + numero2;
};
somaDoisNumeros(10, 20);
```

JS: function expression (funções anônimas)

```
var avg = function () {
    var sum = 0;
    for (var i = 0, j = arguments.length; i < j; i+
+) {
        sum += arguments[i];
    }
    return sum / arguments.length;
}

avg(5,10,5)
6.66666...
```

JS: Template Literals

- Na hora de montar código JS é possível fazer uso do **Template Literals**
  - Este mecanismo ajuda na escrita de código de forma mais fluida

```
let var = `I counted ${3+4} sheep`;
```

- Tudo que estiver dentro do ${ } será interpretado como uma expressão JS.

- ATENÇÃO
  - É usado o character back-tick (`)
  - Se tentar utilizar aspa simples ou dupla, será lido como String.

JS: Spread

- Uso dos 3 pontinhos (...) in JS
- Algumas funções não aceitam que você passe arrays, sendo necessário passar os valores de forma individual
  - Por exemplo, a Math.max()
    - Dentro de um conjunto de valores, irá retornar o maior.
    - Math.max(1, 5, 77, 3, 55) □ Resultado: 77

```
const nums = [1,2,3,4,5]
Math.max(nums) // retorna NaN

Math.max(...nums) // retorna 5
// Mesmo que Math.max(1,2,3,4,5)
```

JS: Spread

- O operador spread (...) permite a iteração (como Array ou String) ser expandida em locais onde múltiplos elementos ou argumentos são esperados.

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];
const mergedArray = [...array1, ...array2];
// Retorna 1 Array:
// [1, 2, 3, 4, 5, 6]

const mergedArray = [array1, array2];
// Retorna 2 Arrays:
// [ [ 1, 2, 3 ], [ 4, 5, 6 ] ]
```

JS: Spread

- Também permite concatenar elementos.

```
let cats = ['cat1', 'cat2'];
let dogs = ['dog1', 'dog2']
allPets = [...cats, ...dogs]
// [ 'cat1', 'cat2', 'dog1', 'dog2' ]
```

- Spread também pode ser usado em objetos, copiando as propriedades de um objeto para outro.

```
felino = {pernas: 4, familia: 'Felidae'}
novoObjeto = {...felino, cor: 'yellow'}
// { pernas: 4, familia: 'Felidae', cor: 'yellow' }
```

JS: Spread

- Também é usado para juntar dois objetos
  - let caninosFelinos = {...caninos, ...felinos}
    - Soma os atributos dos dois objetos em um novo

- Quando há atributos com o mesmo nome, o valor do último parâmetro ganha, pois substitui o anterior.

```
felino = {pernas: 4, familia: 'Felidae'}
canino = {dentes: 36, familia: 'Canidae’}

caninoFelino = {...felino, ...canino}

// { pernas: 4, familia: 'Canidae', dentes: 36 }
```

JS: arrow function

Há uma forma facilitada para declarar funções no JS... As **arrow functions**.

ANTES (MÉTODO TRADICIONAL)

```
var oldWay = function (name, nickname)
{
     return 'My name is ' + nickname +
', ' + name;
};

console.log(oldWay('James Bond',
'Bond'));
// My name is Bond, James Bond
```

DEPOIS (ARROW FUNCTION)

```
let newWay = (name, nickname) => {
     return 'My name is ' + nickname +
', ' + name;
};

console.log(newWay('James Bond',
'Bond'));
// My name is Bond, James Bond
```

CLASSES

JS: Classes Modernas

No JavaScript moderno, você pode criar classes usando a palavra-chave `class`, introduzida no ES6 (ECMAScript 2015).

Elas fornecem uma maneira mais clara e estruturada de criar objetos e trabalhar com herança, tornando o código mais legível.

EXEMPLO DE ESTRUTURA

```
class NomeDaClasse {
// Construtor: inicializa propriedades
constructor(param1, param2) {
        this.propriedade1 = param1;
        this.propriedade2 = param2;
    }
}
```

JS: Classes modernas

```
class NomeDaClasse {
  // Construtor: inicializa propriedades
  constructor(param1, param2) {
    this.propriedade1 = param1;
    this.propriedade2 = param2;
  }
  // Método da instância
  metodoExemplo() {
    console.log(`Propriedade1 é: ${this.propriedade1}`);
  }
  // Método estático (acessado diretamente pela classe, não pela instância)
  static metodoEstatico() {
    console.log("Este é um método estático.");
  }
}

// Criando uma instância da classe
const instancia = new NomeDaClasse("valor1", "valor2");
instancia.metodoExemplo(); // Chamada do método da instância
// Chamando um método estático
NomeDaClasse.metodoEstatico();
```

JS: Classes modernas - Get e Set

```
class Produto {
  #preco; // privado de verdade

  constructor(nome, preco) {
    this.nome = nome;
    this.#preco = preco;
  }

  get preco() {
    return `R$ ${this.#preco.toFixed(2)}`;
  }

  set preco(novoPreco) {
    if (novoPreco > 0) {
      this.#preco = novoPreco;
    } else {
      console.log("O preço deve ser maior que zero.");
    }
  }
}

const produto = new Produto("Notebook", 2500);
console.log(produto.preco);
produto.preco = 3000;
console.log(produto.preco);
```

A depender da versão do JS, usa-se o #variavel para declarar uma variável privada.

Obs.: O Devtools possui uma permissão especial, se tentar acessar uma variável privada diretamente no Devtools você vai conseguir.
Teste em um arquivo HTML.

JS: Classes Modernas - Herança

```
class Animal {
constructor(nome) {
     this.nome = nome;
}
falar() {
          console.log(`${this.nome} faz um som.`);
     }
}

class Cachorro extends Animal {
     falar() {
          console.log(`${this.nome} late.`);
     }
}



// Instanciação e Uso
const dog = new Cachorro("Rex");
dog.falar(); // "Rex late."


const animal = new Animal("Cobra");
animal.falar(); // "Cobra faz um som."
```

Superclasse (Base)

A classe `Animal` define a estrutura base: um construtor para o nome e um método genérico `falar()`.

Subclasse (Herança)

A classe `Cachorro` estende `Animal` usando a palavra-chave **`extends`**.

Polimorfismo

A subclasse **sobrescreve** o método `falar()` para fornecer uma implementação específica.

JS: protótipo

Qualquer atributo ou função adicionado ao protótipo de uma dessas funções ficará disponível em qualquer objeto do tipo gerado por elas.

```
String.prototype.paraNumero = function () {
    if (this == "um") {
        return 1;
    }
}

console.log("um".paraNumero()); // 1
```

JS: protótipo

```
var Pessoa = function (nome, email) {
    this.nome = nome;
    // verifica se o e-mail foi preenchido
    if (email) {
        this.email = email;
    }
}


Pessoa.prototype.email = "contato@ufc.br"
var ricardo = new Pessoa("Ricardo");
console.log(ricardo.email); // contato@ufc.br


var joao = new Pessoa("Joao da Silva",
    "joao@da.silva");
console.log(joao.email); // joao@da.silva
```

JS: protótipo

```
var Pessoa = function (nome, email) {
    this.nome = nome;
    // verifica se o e-mail foi preenchido
    if (email) {
        this.email = email;
    }
};


Pessoa.prototype.fala = function () {
    console.log("Olá, meu nome é " + this.nome + " e meu email é
    “ + this.email);
    };

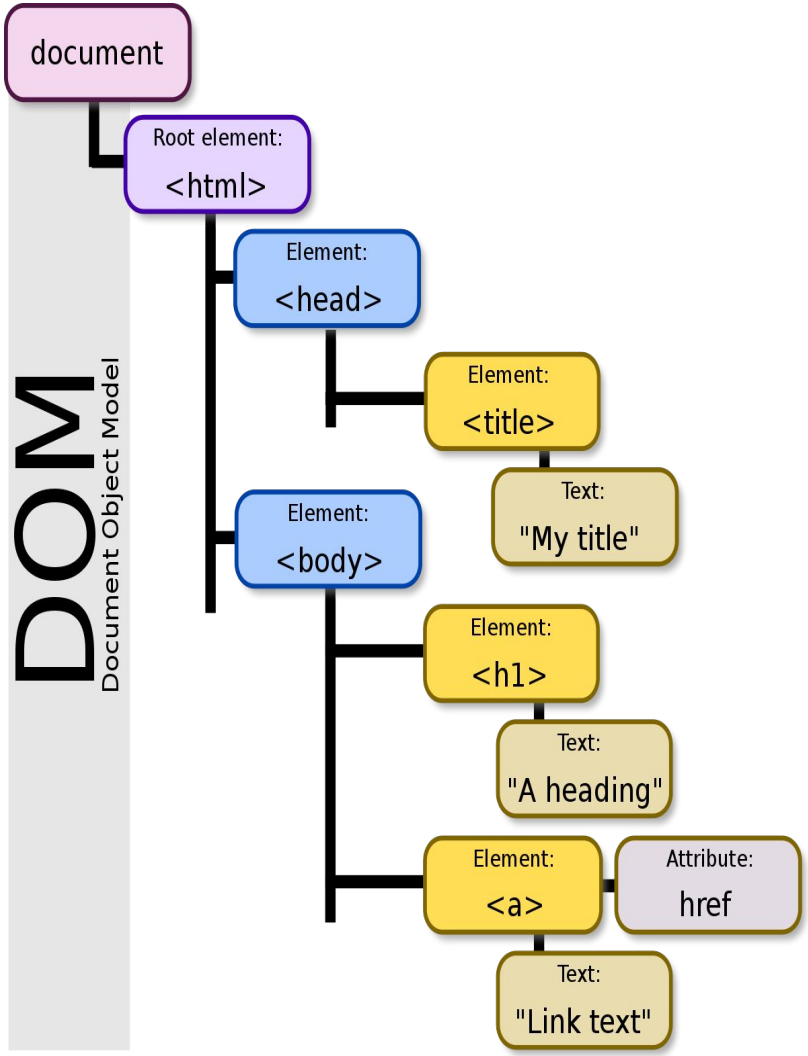

Pessoa.prototype.anda = function () {
    console.log("Estou andando");
};
```

Document Object Model (DOM)

Document Object Model (DOM)

- O Document Object Model (DOM) é uma interface de programação que permite acessar os elementos HTML usando JavaScript. Com o acesso ao componente HTML é possível alterar estrutura, estilo e conteúdo.

- No DOM a página web é representada como nós e objetos.



DOM: Acessando Elementos

Métodos Tradicionais

Seleção por atributos específicos:

- `.getElementById()`
- `.getElementsByClassName()`
- `.getElementsByName()`
- `.getElementsByTagName()`

Seletores Modernos

Seleção flexível via seletores CSS:

- `.querySelector()`
- `.querySelectorAll()`

Estes métodos permitem buscar elementos usando a mesma sintaxe de seletores do CSS.

DOM: getElementById()

```
<p id="tituloPrincipal">Este é o título principal da minha
página</p>

const elem = document.getElementById("tituloPrincipal");
elem.style.color = 'red';
elem.style.backgroundColor = 'blue'
```

ATENÇÃO !!!

Diferente do CSS, o JavaScript não permite hifens (-), por isso, alguns atributos possuem seu nome renomeado.

Por exemplo,

em JS usa-se **backgroundColor** ao invés de **background-color**

DOM: getElementsByClassName()

Permite selecionar todos os elementos que possuem uma ou mais classes.
Eles podem ser acessados como um array.

ESTRUTURA HTML

```
<span class="orange fruit">Orange Fruit</span>

<span class="orange juice">Orange Juice</span>

<span class="apple juice">Apple Juice</span>

<span class="foo bar">Something Random</span>
```

JAVASCRIPT

```
// Somente seleciona os elementos que possuem as duas classes

const allOrangeJuice = document.getElementsByClassName('orange juice');

// Acessando o primeiro item do array resultante

let result = document.getElementsByClassName('orange juice');

result[0].style.color = 'red';
```

DOM: getElementsByName()

Retorna uma lista de elementos com o **atributo name** específico. Útil para capturar grupos de elementos relacionados, como botões de rádio.

```
<!-- HTML: Opções de um formulário -->
<input type="radio" name="genero" value="M" /> Masculino
<input type="radio" name="genero" value="F" /> Feminino

// JavaScript: Capturando todas as opções do grupo
const opcoes = document.getElementsByName("genero");

console.log(opcoes.length); // Resultado: 2
console.log(opcoes[0].type); // Resultado: "radio"

// Iterando sobre a NodeList retornada
opcoes.forEach(opt => console.log(opt.value));
```

DOM: getElementsByTagName()

Permite selecionar todos os elementos que pertencem a determinada **tag HTML**.
Eles podem ser acessados como um array.

```
const todosElementosTagP =
document.getElementsByTagName("p");
const tamanho = todosElementosTagP.length;

console.log(tamanho);

console.dir(todosElementosTagP);

console.log(todosElementosTagP[0].innerText);
```

DOM: querySelector()

Permite selecionar **o primeiro** elemento que satisfaz a query, ou seja, o primeiro elemento que possui o seletor ou grupo de seletores especificados.
Deve-se utilizar a sintaxe do CSS.

```
<h1>Exemplo de site</h1>
<p>Exemplo de tag p do HTML. </p>
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
let minhaPrimeiraTagP = document.querySelector("p");
console.log(minhaPrimeiraTagP);
console.log(minhaPrimeiraTagP.innerText);

let minhaTagPByClass = document.querySelector(".elementop2");
console.log(minhaTagPByClass);
console.log(minhaTagPByClass.innerText);
```

DOM: querySelector()

- Principais propriedades do objeto retornado no querySelector.

PROPERTIES & METHODS

(the important ones)

- classList
- getAttribute()
- setAttribute()
- appendChild()
- append()
- prepend()
- removeChild()
- remove()
- createElement



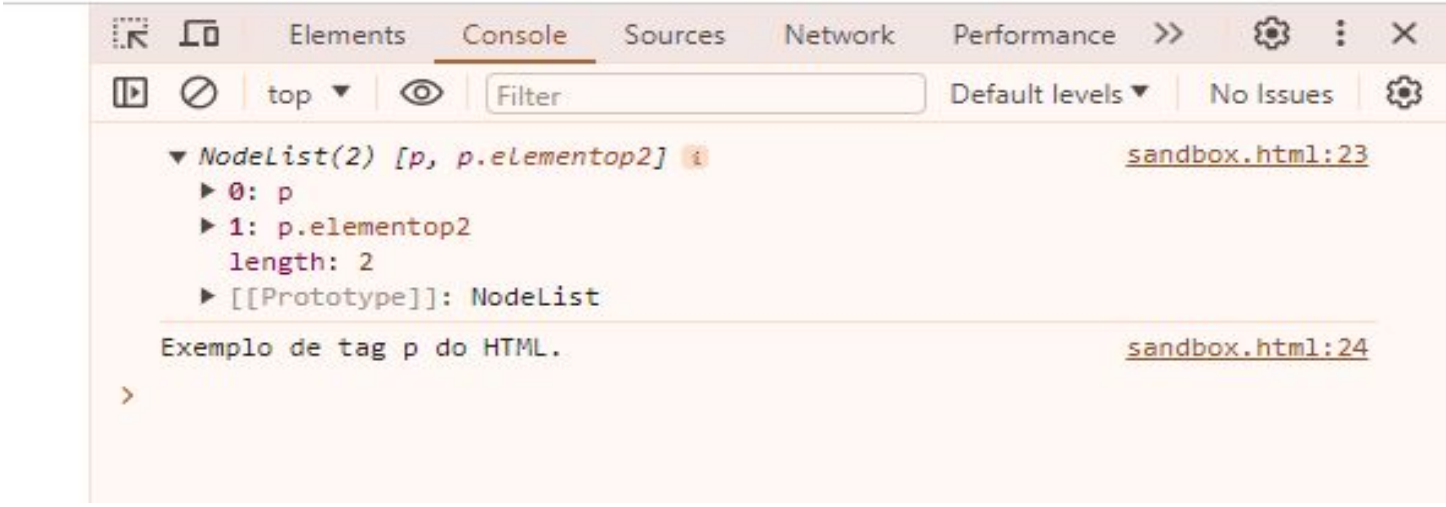
- innerText
- textContent
- innerHTML
- value
- parentElement
- children
- nextSibling
- previousSibling
- style

DOM: querySelectorAll()

Permite selecionar **TODOS** que satisfaz a query.
Deve-se utilizar a sintaxe do CSS.

```
<h1>Exemplo de site</h1>
<p>Exemplo de tag p do HTML. </p>
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
let todasTagP = document.querySelectorAll("p");
console.log(todasTagP);
console.log(todasTagP[0].innerText);
```



DOM: querySelectorAll()

Outro exemplo

```
<p>Exemplo de tag p do HTML. <a href="http://exemplo...">meu
link</a> </p>
<p class="elementop2">Exemplo de tag p do HTML 2. </p>

todosLinksDentroDeP = document.querySelectorAll('p a')
// todos os elementos 'a' que estão dentro de tags 'p'

console.log(todosLinksDentroDeP);
console.log(todosLinksDentroDeP[0].href);
```

E mais um...

```
checkBox =
document.querySelector('input[type="checkbox"]')
//Seleciona o primeiro checkbox da pagina.

console.log(checkBox);
console.log(checkBox.checked);

//Se usar o querySelectorAll, não esquecer de acessar usando arrays.
checkboxs[0].checked.
```

JS: Outras formas de acessar os elementos

- document.getAttribute() e document.setAttribute()
  - É o mesmo que acessar meuObjeto.nomeAtributo.
  - Exemplo:
    - document.querySelector('img').src = 'xxx'
    - é o mesmo que meuObjeto.setAttribute('src', 'xxx').

```
<p class="tagsp">Exemplo de tag p do HTML.</p>
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
// Alterando a classe de um objeto
tagP = document.querySelectorAll('.tagsp')

console.log(tagP);
console.log(tagP.class);

tagP[0].setAttribute('class', 'novaclasse');
novaTagP = document.querySelectorAll('.tagsp')
console.log(novaTagP);

//Antes tinhamos um objeto p com a classe tagsp, agora não temos mais.
```



JS: Outros detalhes...

- Ao alterar o style de um elemento por JavaScript, é como se inseríssemos um *CSS in-line*, ou seja, individual para cada elemento.
  - Mas e se precisarmos redefinir atributos de vários elementos?

- Podemos incluir ou excluir classes de elementos usando o **classList**.
  - Adiciona ou remove classes de um objeto.
  - Ao adicionar uma classe, o elemento herda todas as características da classe.
  - Por exemplo, ".purple" define a cor purple para o objeto.
  - Para adicionar: **h2.classList.add('purple')**
  - Ou **h2.classList.toggle('purple').**
  - O toggle adiciona ou remove, se já tiver ele remove, se não tiver, ele adiciona.

Resumo:
funções, classes, construtores e seleção de elementos.

EXERCÍCIO 1: Classes em JavaScript

Objetivo Geral

Crie uma estrutura de herança onde as classes **Gato**, **Cachorro** e **Pato** herdam da classe base **Animal**.

1. Classe Animal (Base)

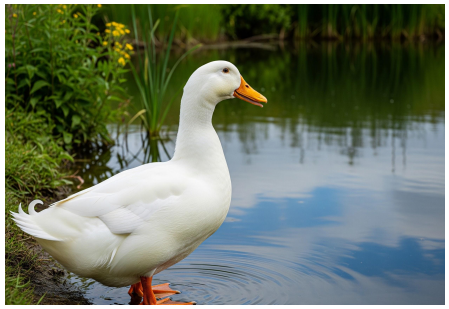
- **Atributos:** nome e tutor.
- **Métodos:** EmitirSom() e Comer().
- **Encapsulamento:** Implemente métodos *Gets* e *Sets* para cada atributo.

2. Implementação e Teste

- **Polimorfismo:** Cada animal deve emitir seu som específico: “Au au”, “Miau” ou “Quack”.
- **Instanciação:** Crie objetos de teste para validar o funcionamento das classes.







EXERCÍCIO 2

- Usando 2 <input>, receber preço e quantidade e calcular o valor total (preço * quantidade) e exibir o valor total abaixo.

- Quando um deles for alterado, alterar automaticamente o valor total.

EXERCÍCIO 3

- Crie uma página web que mostra uma lista de alunos usando <ul> e <li>. Após criar pelo menos 6 nomes de alunos, crie o código CSS e Javascript para alterar a cor do texto para vermelhos dos alunos que possuem uma posição par na lista.

Dica: use o querySelectorAll e o for



Exercício 4: Operador Spread e Arrow Functions

Você está desenvolvendo a tela de "Finalizar Compra" de uma loja virtual. O sistema recebe os dados do usuário, de faturamento e duas listas de produtos (estoque local e parceiros).

Sua tarefa é:

- Unir **`estoqueLocal`** e **`estoqueParceiro`** em **`todosProdutos`** usando Spread.
- Criar **`resumoPedido`** unindo **`cliente`** e **`entrega`** (o CEP de "entrega" deve prevalecer).
- Criar Arrow Function **`gerarRecibo`** que retorne uma string formatada com Template Literals.

Dados iniciais:

```
const cliente = { nome: "Ana Silva", cpf: "111.222.333-44", cep: "00000-000" };

const entrega = { frete: 15.00, cep: "63900-000" };

const estoqueLocal = ["Notebook", "Mouse"];

const estoqueParceiro = ["Teclado Mecânico", "Monitor"];
```

Formato do Recibo: "O cliente [NOME] receberá [QUANTIDADE DE PRODUTOS] itens no CEP [CEP]"

EXERCÍCIO 5: POO E ENCAPSULAMENTO

Uma startup financeira está criando seu novo sistema de contas e você ficou responsável por implementar a classe `ContaCorrente`. O saldo do cliente não pode ser acessado de forma direta para evitar que outros programadores alterem o valor de forma arbitrária.
Sua tarefa é:

01 Criar uma **classe moderna** chamada `ContaCorrente` usando a palavra-chave `class` e o método `constructor` para `titular` e `saldoInicial`.

02 Declarar o saldo como uma **variável privada** (utilizando a sintaxe `#`) para impedir acesso externo direto.

03 Implementar um método `get saldo()` que retorne uma string formatada: *"R$ [VALOR]"*, utilizando o método `.toFixed(2)`.

04 Implementar um método `set saldo(novoValor)` que valide se o valor é **positivo**. Caso contrário, exiba um erro no console.

EXERCÍCIO 6: DOM e JS

Você é o desenvolvedor front-end de uma loja e recebeu a tarefa de ativar o **"Modo Black Friday"** (crie um botão para isso) na página principal via JavaScript. O HTML da sua página possui a seguinte estrutura básica:

```
<!-- Estrutura HTML de base -->
<img id="banner-principal" src="banner-normal.jpg" alt="Promoções">

<div class="produto-card">Smartphone</div>
<div class="produto-card">Notebook</div>
<div class="produto-card">Fone Bluetooth</div>

<p id="cupom-secreto" style="display: none">CUPOM: JSBLACK20</p>
```

Sua tarefa é escrever o código JavaScript que execute os seguintes passos:

1. Troca do Banner (Manipulação de Atributos)

Capture a imagem do banner usando `document.getElementById()`. Utilize `.setAttribute()` para alterar o **src** para "banner-black-friday.jpg".

2. Destaque nos Produtos (Manipulação de Classes)

Use `document.querySelectorAll()` para capturar a classe **produto-card**. Itere sobre a lista (*for* ou *forEach*) e aplique `.classList.add('black-friday-theme')`.

3. Resgate do Cupom (Texto Oculto)

Capture o elemento **#cupom-secreto**. Considerando que está com *display: none*, utilize a propriedade correta (`.innerText` vs `.textContent`) para extrair o texto e exibi-lo no console.

AULA 3/4

JS: eventos

- Exemplos de eventos:
  - Página HTML terminou o carregamento
  - Um elemento HTML foi alterado
  - Um botão HTML foi clicado
  - Etc …

- Com o JS é possível interceptar estes eventos e aplicar alguma ação quando eles acontecerem.

```
<element event='some JavaScript'></element>
```

JS: eventos

```
<!DOCTYPE html>
<html>

<body>

    <button onclick="document.getElementById('demo').innerHTML=Date()">The
 time is?</button>

    <p id="demo"></p>

</body>

</html>
```

```
<!DOCTYPE html>
<html>

<body>

    <button onclick="this.innerHTML=Date()">The time is?</button>

</body>

</html>
```

JS: eventos

- Forma mais elegante...

```
<!DOCTYPE html>
<html>

<body>

    <p>Click the button to display the date.</p>

    <button onclick="displayDate()">The time is?</button>

    <script>
        function displayDate() {
            document.getElementById("demo").innerHTML = Date();
        }
    </script>

    <p id="demo"></p>

</body>

</html>
```

JS: eventos comuns

Evento	Descrição
onchange	Um elemento HTML mudou
onclick	O elemento HTML foi clicado
onmouseover	O mouse passou por cima do elemento HTML
onmouseout	O mouse saiu de cima do elemento HTML
onkeydown	O usuário apertou uma tecla
onload	O navegador terminou de carregar a página

JS: eventos comuns

A SMALL TASTE

- clicks
- drags
- drops
- hovers
- scrolls
- form submission
- key presses
- focus/blur



- mouse wheel
- double click
- copying
- pasting
- audio start
- screen resize
- printing

JS: eventos comuns

```
<body onload="myFunction()">
```

```
<img src="w3javascript.gif" onload="loadImage()"
 width="100" height="132">

<script>
function loadImage() {
  alert("Image is loaded");
}
</script>
```

```
<input type='text' id='num2' onchange="recalcular()”>
```

JS: addEventListener

```html
<button id="meu-botao">Clique em mim</button>
```

```
<script>
    // Passo 1: Pegamos o botão no HTML e guardamos em uma variável
let botao = document.getElementById("meu-botao");

// Passo 2: Adicionamos o ouvinte de eventos (o "fofoqueiro")
botao.addEventListener("click", () => {
    // Passo 3: O que acontece quando o clique ocorrer?
    botao.innerText = "Você clicou no botão!";
    botao.style.backgroundColor = "green";
    botao.style.color = "white";
});
</script>
```

JS: addEventListener

```
// --- FUNÇÃO 2: Focada em emitir avisos ---
botao.addEventListener("click", () => {
    console.log("O botão foi clicado! Registrando no sistema...");
    alert("Esta é a segunda função rodando no mesmo clique!");
});
```

O addEventListener permite associar mais de uma função ao mesmo evento.
Por exemplo, fun1 e fun2, usando somente o onclick = fun1, nunca irá chamar as duas funções, só se colocássemos elas aninhadas.

Com o addEventListener, cada chamada ao método é um novo evento/função adicionado.

Como terceiro parâmetro, também é possível passar “options”.
Por exemplo, {once: true} vai permitir que a função execute somente 1x, após isso, é removida da lista de funções a serem executadas no evento.

JS: BOAS PRÁTICAS

- Evite usar variáveis globais
  - Minimize o uso de variáveis globais.
  - Evite também objetos e funções globais.
  - Variáveis globais podem ser sobrescritas por outros trechos de código, levando ao caos.
- Use variáveis locais
  - Variáveis locais devem ser declaradas com var ou let, caso contrário, serão variáveis globais.
    - var tem escopo de função
    - let tem escopo de bloco

JS: BOAS PRÁTICAS

```
<script>
    var  x = 10;
    // Here x is 10
{
  var x = 2;
  // Here x is 2
}
// Here x is 2
</script>
```

```
<script>
    var  x = 10;
    // Here x is 10
{
        let x = 2;
  // Here x is 2
}
// Here x is 10
</script>
```

JS: BOAS PRÁTICAS

- Coloque todas as declarações no começo de cada trecho de código
  - Código limpo e claro
  - Local único para saber quais variáveis há
  - Fácil identificar o uso de variáveis globais
  - Reduz o risco de reescrever uma variável

JS: BOAS PRÁTICAS

```
// Declare no começo
var firstName, lastName, price, discount, fullPrice;

// Use depois
firstName = "John";
lastName = "Doe";

price = 19.90;
discount = 0.10;

fullPrice = price * 100 / discount;

// TAMBÉM VALE PARA LOOPS. Declare no começo.
var i;

// Use depois
for (i = 0; i < 5; i++) {}
```

JS: BOAS PRÁTICAS

- Inicialize as variáveis após a declaração
  - Provê código limpo
  - Evita valores undefined
  - Único lugar para consultar as inicializações

```
// Declare e inicialize no início
var firstName = "",
    lastName = "",
    price = 0,
    discount = 0,
    fullPrice = 0,
    myArray = [],
    myObject = {};
```

JS: BOAS PRÁTICAS

- Não declare tipos básicos com tipos complexos
  - Exemplo: não use new String(“”), isso irá criar um Object
- Use {} ao invés de new Object()
- Use "" ao invés de new String()
- Use 0 ao invés de new Number()
- Use false ao invés de new Boolean()
- Use [] ao invés de new Array()
- Use function (){} ao invés de new Function()

```
var x1 = {};           // new object
var x2 = "";           // new primitive string
var x3 = 0;            // new primitive number
var x4 = false;        // new primitive boolean
var x5 = [];           // new array object
var x6 = /()/;         // new regexp object
var x7 = function () { }; // new function object
```

JS: BOAS PRÁTICAS

- Cuidado com as conversões de String e Number

```
var x = 5 + 7;       // x.valueOf() is 12,  typeof x is a number
var x = 5 + "7";     // x.valueOf() is 57,  typeof x is a string
var x = "5" + 7;     // x.valueOf() is 57,  typeof x is a string
var x = 5 - 7;       // x.valueOf() is -2,  typeof x is a number
var x = 5 - "7";     // x.valueOf() is -2,  typeof x is a number
var x = "5" - 7;     // x.valueOf() is -2,  typeof x is a number
var x = 5 - "x";     // x.valueOf() is NaN, typeof x is a number
```

JS: BOAS PRÁTICAS

- Use === ao invés de ==
  - O operador == converte automaticamente os valores para o mesmo tipo antes de verificar
  - O operador === força que os tipos sejam iguais

```
0 == "";        // true
1 == "1";       // true
1 == true;      // true

0 === "";       // false
1 === "1";      // false
1 === true;     // false
```

JS: BOAS PRÁTICAS

- Use valores default para suas variáveis, um valor undefined pode quebrar o seu programa

```
function myFunction(x, y) {
    if (y === undefined) {
        y = 0;
    }
}

// OU

function (a = 1, b = 1) { /*function code*/ }
```

JS: BOAS PRÁTICAS

- Sempre termine os Switches com o valor default

```
switch (new Date().getDay()) {
    case 0:
        day = "Sunday";
        break;
    case 1:
        day = "Monday";
        break;
    case 2:
        day = "Tuesday";
        break;
    case 3:
        day = "Wednesday";
        break;
    case 4:
        day = "Thursday";
        break;
    case 5:
        day = "Friday";
        break;
    case 6:
        day = "Saturday";
        break;
    default:
        day = "Unknown";
}
```

JS: BOAS PRÁTICAS

- Evite usar a função eval()
  - Esta função serve para executar código texto como código JS, pode levar a diversos problemas de segurança...
- Reduza a quantidade de comandos dentro dos loops

```
var i;
for (i = 0; i < arr.length; i++) { }

//Melhor usar assim:
var i;
var l = arr.length;
for (i = 0; i < l; i++) {?
```

JS: eval()

```
let textoDigitado = "10 * 5 + 2";
// O eval lê o texto, percebe que é uma conta matemática
e executa!
let resultado = eval(textoDigitado);
console.log(resultado); // Vai imprimir: 52
```

Exemplo: Calculadora com eval()

```
let textoDigitado = "document.body.innerHTML = '<h1>VOCÊ
FOI HACKEADO</h1>';";
// O seu sistema confia na entrada e roda o eval:
eval(textoDigitado);
```

JS: eval()

```
let textoDigitado = "10 + 5";
// Cortamos o texto onde tem o "+"
let partes = textoDigitado.split("+");
// Agora 'partes' é uma lista (array) com dois textos: ["10", " 5"]
// Transformamos os textos em números reais e somamos de verdade
let numero1 = parseFloat(partes[0]); // 10
let numero2 = parseFloat(partes[1]); // 5
let resultado = numero1 + numero2;
console.log(resultado); // Imprime: 15
```

JS: Exercício

- Criar uma página HTML com somente 1 botão.
- Ao clicar no botão, deve-se alterar o background-color usando uma cor RGB aleatória.
- Para gerar números aleatórios no intervalo de 0-255
  - Math.floor(Math.random() * 255); //Math.random() retorna número entre 0 e 1, ao multiplicar por 255 temos o range desejado.



JS: Exercício

- Criar uma página Pokémon! A cada clique na página (em qualquer lugar) deve ser inserido um novo Pokémon.
- No exemplo abaixo, cliquei na página 9 vezes.


#1


#2


#3


#4


#5


#6


#7


#8


#9

JS: Exercício

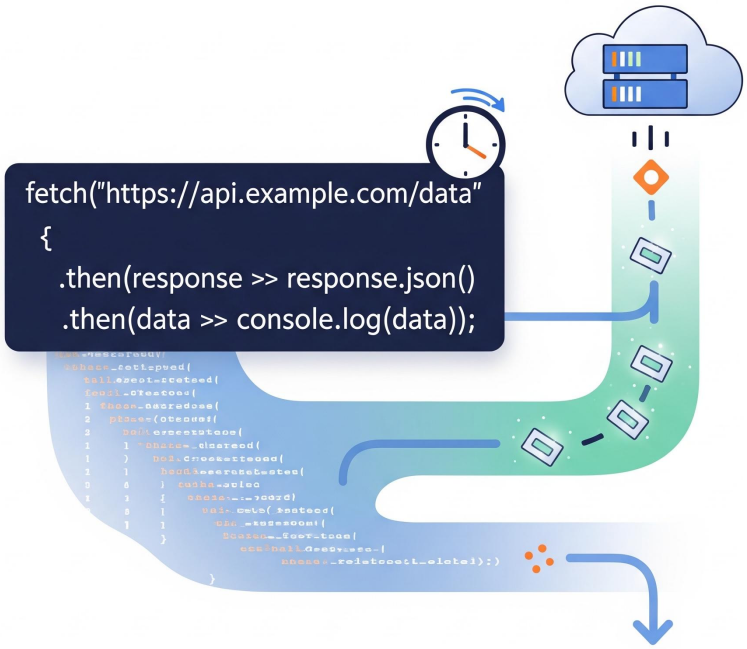
- Crie uma calculadora para as 4 operações básicas, soma, subtração, divisão e multiplicação.
- NÃO USE EVAL OU BIBLIOTECAS EXTERNAS.
- Faça como no exemplo, quebre a entrada em n partes, percorra com um laço e realize a operação de cada parte.
- Ignore parênteses ou operações muito complexas.

Dúvidas?



AULA 4/4

JS: Promises



O que é?

Um objeto que representa o sucesso ou a falha eventual de uma operação assíncrona.

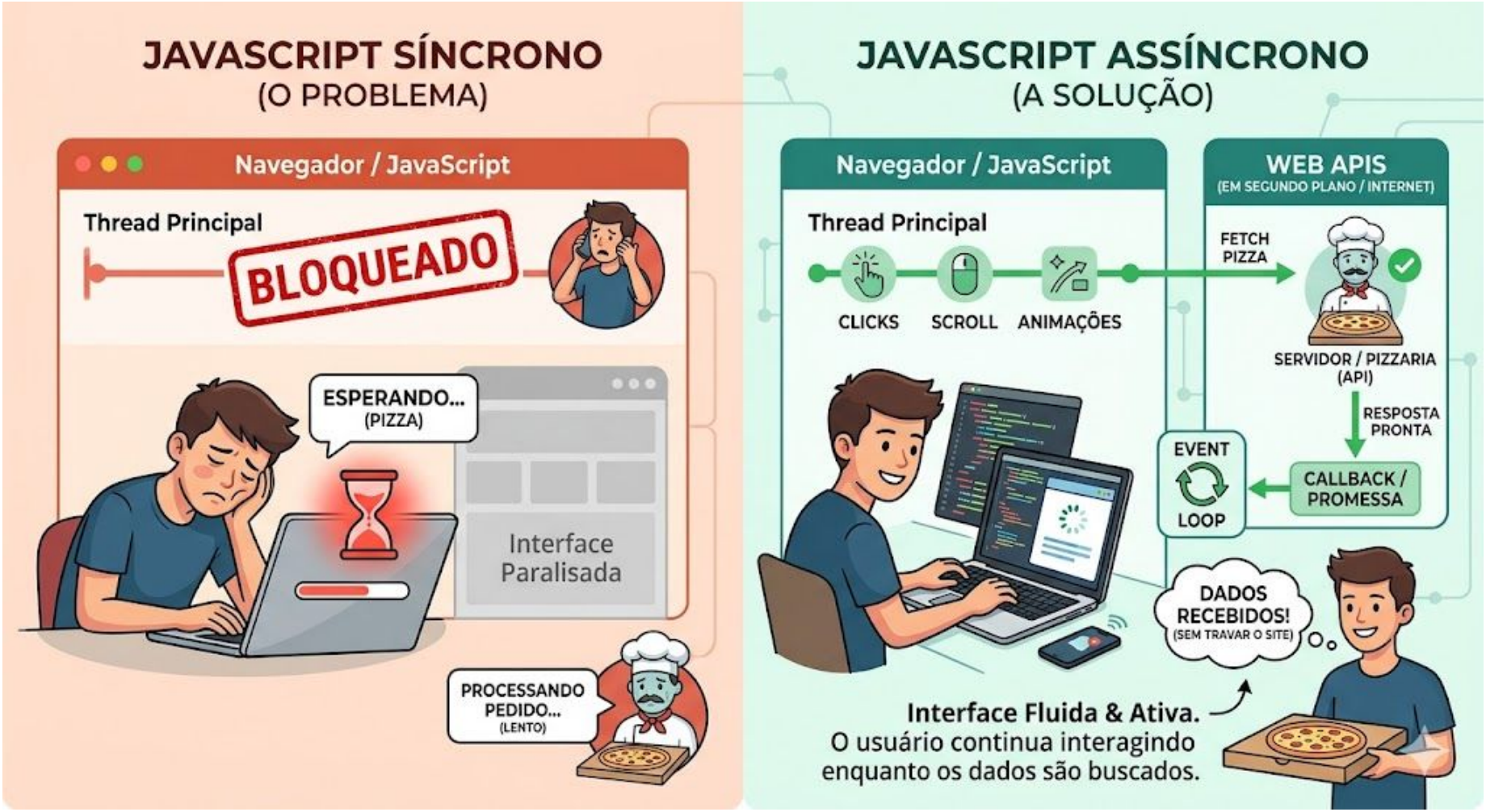
Como funciona?

- **Pendência:** A operação ainda não terminou.
- **Resolvida:** Sucesso! O valor está disponível.
- **Rejeitada:** A operação falhou.

Vantagem

Permite anexar "handlers" (then/catch) para lidar com o resultado sem travar o código.

JS: Promises


JAVASCRIPT SÍNCRONO
(O PROBLEMA)
Navegador / JavaScript
Thread Principal
BLOQUEADO
ESPERANDO...
(PIZZA)
Interface
Paralisada
PROCESSANDO
PEDIDO...
(LENTO)
JAVASCRIPT ASSÍNCRONO
(A SOLUÇÃO)
Navegador / JavaScript
Thread Principal
CLICKS
SCROLL
ANIMAÇÕES
WEB APIS
(EM SEGUNDO PLANO / INTERNET)
FETCH
PIZZA
SERVIDOR / PIZZARIA
(API)
RESPOSTA
PRONTA
CALLBACK /
PROMESSA
EVENT
LOOP
DADOS
RECEBIDOS!
(SEM TRAVAR O SITE)
Interface Fluida & Ativa.
O usuário continua interagindo
enquanto os dados são buscados.

JS: Promises Exemplo de Uso

- Faremos uma solicitação HTTP para o servidor.
- Em uma solicitação HTTP, enviamos uma mensagem para um servidor remoto, e ele nos retorna uma resposta.
- Neste caso, enviaremos uma solicitação para obter um arquivo JSON do servidor.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

console.log(fetchPromise);

fetchPromise.then((response) => {
  console.log(`Received response: ${response.status}`);
});

console.log("Started request…");
```

1. Chamando a API **fetch()** e atribuindo o valor retornado à variável `fetchPromise`.
2. Registrando no console o valor da variável `fetchPromise`. Isso deve exibir algo como: `Promise { <state>: "pending" }`, indicando que temos um objeto **Promise**, e ele possui um estado com valor "pending". O estado "pending" significa que a operação de fetch ainda está em andamento.
3. Passando uma função manipuladora para o método `then()` da **Promise**. Quando (e se) a operação de fetch for bem-sucedida, a promise chamará nosso manipulador, passando um objeto **Response**, que contém a resposta do servidor.
4. Registrando uma mensagem no console indicando que a solicitação foi iniciada.

JS: Promises Exemplo de Uso

- Com a API fetch(), assim que você obtém um objeto Response, é necessário chamar outra função para obter os dados da resposta.

- Neste caso, queremos obter os dados da resposta como JSON, então chamamos o método json() do objeto Response.

- Acontece que o método json() também é assíncrono. Portanto, este é um caso em que precisamos chamar duas funções assíncronas sucessivas.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise.then((response) => {
  const jsonPromise = response.json();
  jsonPromise.then((data) => {
    console.log(data[0].name);
  });
});
```

- Neste exemplo, como antes, adicionamos um manipulador then() à promise retornada pelo fetch(). Mas, desta vez, nosso manipulador chama response.json() e, em seguida, passa um novo manipulador then() para a promise retornada por response.json().
- Isso deve registrar no console "baked beans" (o nome do primeiro produto listado em "products.json").

JS: Promises Exemplo de Uso

- Lembra quando dissemos que ao chamar um callback dentro de outro callback acabávamos criando níveis sucessivamente mais aninhados de código?

- E dissemos que esse "inferno dos callbacks" tornava nosso código difícil de entender? Isso não é exatamente a mesma coisa, só que com chamadas ao then()?

- De fato, é. Mas a característica elegante das promises é que o próprio then() retorna uma promise, que será resolvida com o resultado da função passada para ele.

- Isso significa que podemos (e certamente devemos) reescrever o código.

JS: Promises Exemplo de Uso

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise
  .then((response) => response.json())
  .then((data) => {
    console.log(data[0].name);
  });
```

- Em vez de chamar o segundo then() dentro do manipulador do primeiro then(), podemos retornar a promise retornada por json() e chamar o segundo then() nesse valor de retorno.

- Isso é chamado de encadeamento de promises (promise chaining) e nos permite evitar níveis cada vez maiores de indentação quando precisamos fazer chamadas consecutivas a funções assíncronas.

JS: Promises Exemplo de Uso

Por fim …

```
const fetchPromise = fetch(
  "bad-scheme://mdn.github.io/learning-area/javascript/apis/fetching-data/can-
store/products.json",
);

fetchPromise
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    return response.json();
  })
  .then((data) => {
    console.log(data[0].name);
  })
  .catch((error) => {
    console.error(`Could not get products: ${error}`);
  });
```

JS: Async Await

- Há uma forma ainda mais elegante de trabalhar com código assíncrono no Javascript, ao invés de usar a notação convencional para Promises, vamos utilizar o padrão Async e Await.

- A palavra-chave async oferece uma maneira mais simples de trabalhar com código assíncrono baseado em promises. Adicionar async no início de uma função a torna uma função assíncrona:

```
async function myFunction() {
  // This is an async function
}
```

JS: Async Await

- Dentro de uma função async, você pode usar a palavra-chave await antes de uma chamada para uma função que retorna uma promise.

- Isso faz com que o código espere nesse ponto até que a promise seja resolvida, momento em que o valor resolvido da promise é tratado como um valor de retorno, ou o valor rejeitado é lançado como um erro.

- Isso permite que você escreva código que utiliza funções assíncronas, mas que se parece com código síncrono. Por exemplo, poderíamos usá-la para reescrever nosso exemplo de fetch.

Exemplos de uso

- https://hp-api.onrender.com/
- API IBGE
- API Prefeitura Quixadá

```
async function fetchAllCharacters() {
  try {
    const response = await fetch('https://hp-api.onrender.com/api/characters');
    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }
    const characters = await response.json();
    console.log("Personagens:", characters);
    return characters;
  } catch (error) {
    console.error("Falha ao buscar personagens:", error);
  }
}

async function getDataa(){
  const allChars = await fetchAllCharacters();
  console.log("Primeiro personagem:", allChars[0].name);
}

getDataa();
```

```
async function fetchCharacterById(id) {
  try {
    const response = await fetch(`https://hp-api.onrender.com/api/character/${id}`);
    if (!response.ok) {
      throw new Error(`Personagem não encontrado (ID: ${id})`);
    }
    const character = await response.json();
    console.log("Personagem encontrado:", character);
    return character;
  } catch (error) {
    console.error("Erro:", error.message);
  }
}

async function getData(){
  const char = await fetchCharacterById("9e3f7ce4-b9a7-4244-b709-dae5c1f1d4a8"); // ID de
Harry Potter
  console.log(char[0].name);
}

getData();
```

```
async function fetchGryffindorStudents() {
  try {
    const response = await
fetch('https://hp-api.onrender.com/api/characters/house/gryffindor');
    const students = await response.json();
    console.log("Alunos da Grifinória:", students);

    // Filtra apenas alunos vivos (opcional)
    const livingStudents = students.filter(student => !student.alive);
    console.log("Alunos vivos:", livingStudents);
  } catch (error) {
    console.error("Erro ao buscar alunos:", error);
  }
}

// Uso
fetchGryffindorStudents();
```

```
async function fetchSpells() {
  try {
    const response = await fetch('https://hp-api.onrender.com/api/spells');
    const spells = await response.json();

    console.log("Feitiços disponíveis:");
    spells.forEach(spell => {
      console.log(`- ${spell.name}: ${spell.description}`);
    });

    return spells;
  } catch (error) {
    console.error("Erro ao buscar feitiços:", error);
  }
}

// Uso
fetchSpells();
```

JS: Async Await - Atividade

. Crie um sistema para baixar os dados da URL abaixo e exibir, em formato de card, os dados dos personagens, inclusive a imagem.
. Se o personagem não tiver imagem, exiba uma imagem padrão.

URL com os dados: https://hp-api.onrender.com/api/characters/staff

Dúvidas?



Projeto Final - JavaScript

- CRIAR UMA PAGINA QUE A CADA CLIQUE ADICIONA UM POKEMON

```
// https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1.png

const container = document.querySelector('#container');
const baseURL =
'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/'


for (let i = 1; i <= 151; i++) {
    const pokemon = document.createElement('div');
    pokemon.classList.add('pokemon');
    const label = document.createElement('span');
    label.innerText = `#${i}`;
    const newImg = document.createElement('img');
    newImg.src = `${baseURL}${i}.png`


    pokemon.appendChild(newImg);
    pokemon.appendChild(label);
    container.appendChild(pokemon)
}
```

```
.pokemon {
    display: inline-block;
    text-align: center;
}
.pokemon img {
    display: block;
}
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1
    <title>Pokemon</title>
    <link rel="stylesheet" href="app.css">
</head>

<body>
    <h1>Look at my pokemon!</h1>
    <section id="container"></section>
    <script src="app.js"></script>
</body>
```